Chapter 8. Difference-in-Differences

After discussing treatment effect heterogeneity, it's time to switch gears a bit, back into average treatment effects. Over the next few chapters, you'll learn how to leverage *panel data for causal inference*.

A panel is a data structure that has repeated observations across time. The fact that you observe the same unit in multiple time periods allows you to see, for the same unit, what happens before and after a treatment takes place. This makes panel data a promising alternative to identifying the causal effects when randomization is not possible. When you have observational (nonrandomized) data and the likely presence of unobserved confounders, panel data methods are as good as it gets in terms of properly identifying the treatment effect.

In this chapter, you'll see why panel data is so interesting for causal inference. Then, you'll learn the most famous causal inference estimator for panel data: difference-in-differences—and many variations of it. To keep things interesting, you'll do all of this in the context of figuring out the effect of an offline marketing campaign.

DATA REGIMES

In contrast to *panel data* or longitudinal design, *cross-sectional* data is characterized by each unit appearing only once. A third category, which falls between the two, is known as *repeated cross-sectional data*. This type of data involves multiple time entries, but the units in each entry are not necessarily the same. Up until this point, you have worked with data that includes repeated observations of the same unit over time (for example, when trying to determine the effect of discounts on restaurant sales), but for the sake of simplicity, we treated that data as cross-sectional. This is sometimes referred to as *pooled cross-section*.

Panel Data

To motivate the use of panel data, I'll mostly talk about causal inference applications to marketing. Marketing is particularly interesting for its notorious difficulty in running randomized experiments. In marketing, you often can't control who receives the treatment, that is, who sees your advertisements. When a new user comes to your site or downloads your app, you have no good way of knowing if that user came because they saw one of your campaigns or due to some other reason. Even if you know that the customer clicked one of your marketing links, it's hard to tell if they wouldn't have gotten your product regardless. For example, if the customer clicked your sponsored Google link, they might just as well have scrolled down a bit and clicked the unpaid link, if they were really looking for your product.

The problem is even bigger with offline marketing. How can you know if placing some billboards in a city brings value in excess of its costs? Because of that, a common practice in marketing is to run geo-experiments: you can deploy a marketing campaign to some geographical region but not others and compare them. In this design, panel data methods are particularly interesting: you can collect data on an entire geography (unit) across multiple periods of time.

Like I've said, panel data is when you have multiple units i over multiple periods of time t. In some marketplace websites, units might be the person and t the days or months. But the unit doesn't need to be a single customer. For example, in the context of an offline marketing campaign, i could be cities where you can place a billboard for your product.

So you can follow along with something more tangible, the following data frame, `mkt_data`, has marketing data in a panel format. Each line is a (day, city) combination:

```
In [1]: import pandas as pd
        import numpy as np

        mkt_data = (pd.read_csv("./data/short_offline
                    .astype({"date":"datetime64[ns]"}

        mkt_data.head()
```

	date	city	region	t
0	2021-05-01	5	S	0
1	2021-05-02	5	S	0
2	2021-05-03	5	S	0
3	2021-05-04	5	S	0
4	2021-05-05	5	S	0

This data frame is sorted by date and city. The outcome variable you care about is number of downloads. Since t will be used to represent time, to avoid confusion, from now on, I'll use D to denote the treatment. Also, in the panel data literature, the treatment is often referred to as an intervention. I'll use both terms interchangeably. In this example, the marketing team launched an offline campaign on the cities with $D_i = 1$. As for the time dimension, let's establish that

T will be the number of periods, with T_{pre} being the periods before the intervention. You can think about the time vector as $t = \{1, 2, \ldots, T_{pre}, T_{pre} + 1, \ldots, T\}$. Periods after the treatment, $T_{pre}, \ldots, T$, are conveniently called post intervention. To simplify the notation, I'll often use a $Post$ dummy, which is 1 when $t > T_{pre}$ and 0 otherwise.

The intervention only happens to treated units, $D = 1$, at the post-intervention period, $t > T_{pre}$. The combination of treatment and post intervention will be denoted by $W = D * \mathbb{1}(t > T_{pre})$ or $W = D * Post$. Here is an example of what this looks like in the marketing data:

```
In [2]: (mkt_data
          .assign(w = lambda d: d["treated"]*d["post"]
          .groupby(["w"])
          .agg({"date":[min, max]}))
```

	date	
	min	**max**
w		
0	2021-05-01	2021-06-01
1	2021-05-15	2021-06-01

As you can see, the pre-intervention period is from 2021-05-01 to 2021-05-15 and the post-intervention period, from 2021-05-15 to 2021-06-01.

This dataset also has a τ variable to denote the treatment effect. Since this data is simulated, I know exactly what that effect is. I've included it in this dataset just for you to check if the methods you'll learn about are doing a good job in identifying the causal effect. But don't get used to it. In real life, you won't have this luxury.

Now that you understand the data better and have learned a new bit of technical notation, you can restate your goal more precisely. You want to understand the effect of the offline marketing campaign on the cities that got treated, after the treatment takes place:

$$ATT = E[Y_{it}(1) - Y_{it}(0)|D = 1, t > T_{pre}]$$

This is the ATT since you are only focusing on understanding the impact the campaign had on the cities with $D = 1$, after the campaign was launched $t > T_{pre}$. Since $Y_{it}(1)$ is observable, you can achieve this goal by imputing the missing potential outcome $E[Y(0)|D = 1, Post = 1]$.

Figure 8-1 shows why panel data becomes particularly interesting when you represent the observed outcomes in a unit-by-time matrix. This matrix highlights the fact that $Y(1)$ is only observable for treated units during the post-treatment period, while for all other cells, you can observe $Y(0)$. Despite this, these cells can still be useful for estimating the missing potential outcome $E[Y(0)|D = 1, t > T_{pre}]$. You can leverage the correlation between units by using the outcome of the control units in the post-intervention period, and you can also leverage correlation across time by using the treated units' outcome in the pre-treatment period.

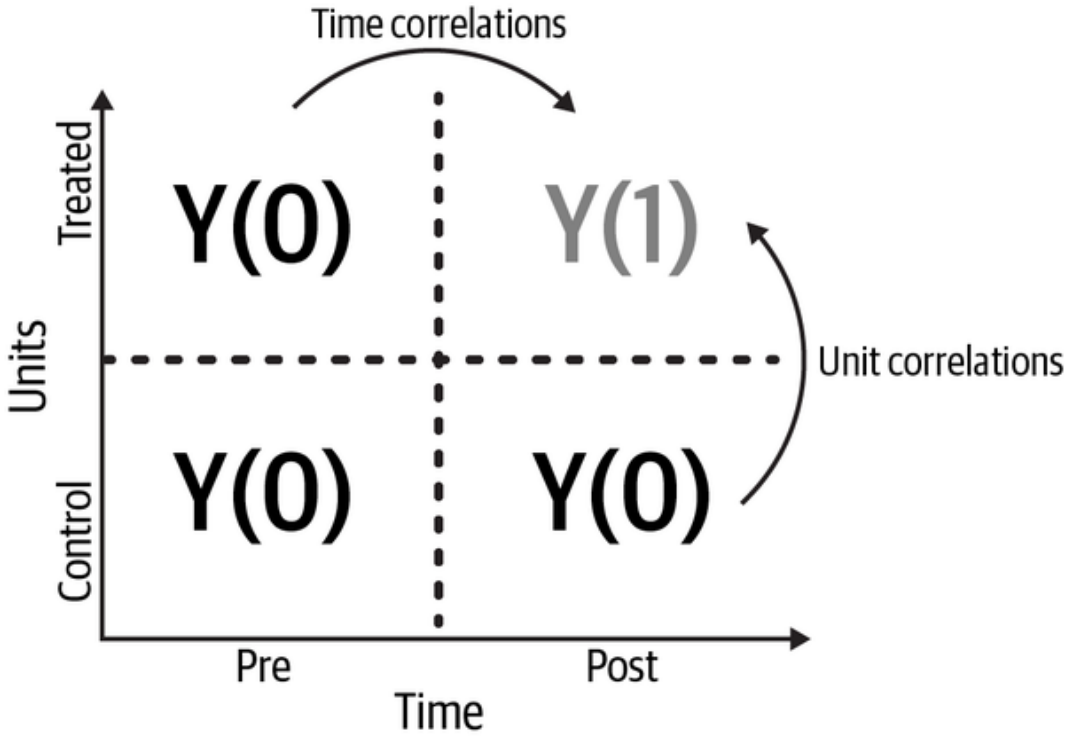


Figure 8-1. By observing the same units across multiple time periods, panel data allows you to leverage the correlation between units and across time to impute the missing potential outcome $T(1)$

Figure 8-1 also shows why you should focus on ATT in most applications with panel data: it's much easier to impute $Y(0)$ for the treated units. If instead you wanted the ATC (average effect on the control), you would have to impute $Y(1)$. However, you would only have one cell where that potential outcome is observable.

Now that you had your brief introduction to panel data, it's time to explore some of the machinery that leverages it to identify and estimate the treatment effect.

Canonical Difference-in-Differences

The basic idea behind difference-in-differences is to impute the missing potential outcome $E[Y(0)|D=1, Post=1]$ by using the

baseline from the treated units, but applying the evolution of the outcome (growth) from the control units:

$$\begin{aligned} E[Y(0)|D=1, Post=1] &= E[Y|D=1, Post=0] \\ &+(E[Y|D=0, Post=1] - E[Y|D=0, Post=0]) \end{aligned}$$

where you can estimate $E[Y(0)|D=1, Post=1]$ by replacing the righthand side expectations with sample averages. The reason this is called the difference-in-differences (DID) estimator is because, if you substitute the preceding expression for $E[Y(0)|D=1, Post=1]$ in the ATT, you get, quite literally, the difference in differences:

$$\begin{aligned} ATT &= (E[Y|D=1, Post=1] - E[Y|D=1, Post=0]) \\ &-(E[Y|D=0, Post=1] - E[Y|D=0, Post=0]) \end{aligned}$$

Don't let all those expectations scare you. In its canonical form, you can get the DID estimate quite easily. First, you divide the time periods in your data into pre- and post-intervention. Then, you divide the units in a treated and control group. Finally, you can simply compute the averages of all the four cells: pre-treatment and control, pre-treatment and treated, post-treatment and control, and post-treatment and treated:

```
In [3]: did_data = (mkt_data
                    .groupby(["treated", "post"])
                    .agg({"downloads":"mean", "date":

        did_data
```

		downloads	date
treated	**post**		
0	**0**	50.335034	2021-05-01
	1	50.556878	2021-05-15
1	**0**	50.944444	2021-05-01
	1	51.858025	2021-05-15

Those are all the numbers you need to get the DID estimate. For the treatment baseline, $E[Y|D = 1, Post = 0]$, you can index into the treatment with `did_data.loc[1]` and then into the pre-treatment period with a follow up `.loc[0]`. To get the evolution in the outcome for the control, $E[Y|D = 0, Post = 1] - E[Y|D = 0, Post = 0]$, you can index into the control with `did data.loc[0]`, compute the difference with `.diff()`, and index into the last row with a follow-up `.loc[1]`. Adding the control trend into the treated baseline gives you an estimate for the counterfactual $E[Y(0)|D = 1, Post = 1]$. To get the ATT, you can subtract that from the average outcome of the treated in the post-intervention period:

```
In [4]: y0_est = (did_data.loc[1].loc[0, "downloads"]
                  # control evolution
                  + did_data.loc[0].diff().loc[1, "do
```

```
att = did_data.loc[1].loc[1, "downloads"] - y
att
```

```
Out[4]: 0.6917359536407233
```

If you compare this number with the true ATT (filtering the treated units and the post-treatment period), you can see that the DID estimate is quite close to what it tries to estimate:

```
In [5]: mkt_data.query("post==1").query("treated==1")
```

```
Out[5]: 0.7660316402518457
```

RACTICAL EXAMPLE

MINIMUM WAGES AND EMPLOYMENT

In the ’90s, David Card and Alan Krueger used a 2 × 2 DID to challenge the conventional economic theory that states that a rise in minimum wage leads to a decrease in employment. They looked at data from fast-food restaurants in New Jersey and Pennsylvania, before and after an increase in New Jersey’s minimum wage. The study found no evidence of reduced employment due to the minimum wage increase. This paper was incredibly influential and got revisited many times and that result proved to be very robust. Eventually, due to its influence and for helping to popularize DID, Card was awarded the Nobel Prize in 2021.

Diff-in-Diff with Outcome Growth

Another very interesting take on DID is to realize it is differentiating the data in the time dimension. Let’s define the difference in the outcome across time for unit i as $\Delta y_i = E[y_i|t > T_{pre}] - E[y_i|t \leq T_{pre}]$. Now, let’s convert your original data, which was by time and unit, into a data frame with Δy_i, where the time dimension has been differentiated out:

```
In [6]: pre = mkt_data.query("post==0").groupby("city
        post = mkt_data.query("post==1").groupby("cit

        delta_y = ((post - pre)
                   .rename("delta_y")
                   .to_frame()
                   # add the treatment dummy
                   .join(mkt_data.groupby("city")["tr
```

```
delta_y.tail()
```

	delta_y	treated
city		
192	0.555556	0
193	0.166667	0
195	0.420635	0
196	0.119048	0
197	1.595238	1

Next, you can use potential outcome notation to define the ATT in terms of Δy:

$$ATT = E[\Delta y_1 - \Delta y_0],$$

which DID tries to identify by replacing Δy_0 with the average of the control units:

$$ATT = E[\Delta y | D = 1] - E[\Delta y | D = 0]$$

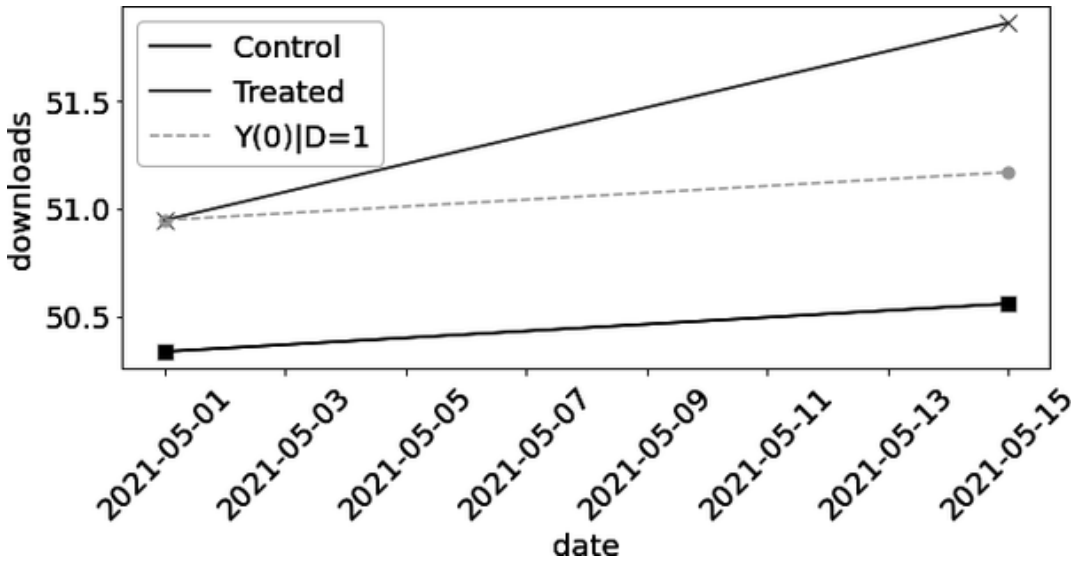
If you replace those expectations with sample averages, you'll see that you get back the same DID estimate you got before:

```
In [7]: (delta_y.query("treated==1")["delta_y"].mean(
          - delta_y.query("treated==0")["delta_y"].mea
```

```
Out[7]: 0.6917359536407155
```

This is an interesting take on DID because it makes very clear what it is assuming, that is, $E[\Delta y_0] = E[\Delta y|D = 0]$, but we'll talk more about this later.

Since this has all been very technical and full of math, I wanted to give you a more visual understanding of DID by plotting the observed outcomes of the treated and control group over time, alongside the estimated counterfactual outcome for the treated unit. In the following image, the DID estimate for $E[Y(0)|D = 1]$ is shown as a dashed line. It was obtained by applying the trajectory from the control into the treatment baseline. The estimated ATT would then be the difference between the estimated counterfactual outcome $Y(0)$ and the observed outcome $Y(1)$, both in the post-treatment period (difference between the dot and the cross):



Diff-in-Diff with OLS

Even though you can implement DID by hand, computing averages or taking deltas, this wouldn't be a respectable causal inference chapter if it didn't include a fair amount of linear regression. Not surprisingly, you can get the exact same DID estimator with a saturated regression model. First, let's group your daily data by city and period—post- and pre-treatment. Then, for each city and period combination, you can get the average number of daily downloads. I'm also getting the start date for each period and the treatment status for each city. The start date isn't used in the estimator, but it's good for understanding when the treatment takes place:

```
In [8]: did_data = (mkt_data
                    .groupby(["city", "post"])
                    .agg({"downloads":"mean", "date":
                    .reset_index())

        did_data.head()
```

	city	post	downloads	d
0	5	0	50.642857	2
1	5	1	50.166667	2
2	15	0	49.142857	2
3	15	1	49.166667	2
4	20	0	48.785714	2

With this city by period dataset, you can estimate the following linear model:

$$Y_{it} = \beta_0 + \beta_1 D_i + \beta_2 Post_t + \beta_3 D_i Post_t + e_{it}$$

and the parameter estimate $\widehat{\beta_3}$ will be the DID estimate. To see why that is, notice that β_0 is the baseline of the control. In this case, β_0 is the level of downloads in control cities, prior to 2021-05-15. If you turn on the treated city dummy, you get $\beta_0 + \beta_1$. So $\beta_0 + \beta_1$ is the baseline of treated cities, also before the intervention. β_1 is simply the difference in baseline between treated and control cities. If you turn the treatment dummy off and turn the post-treatment dummy on, you get $\beta_0 + \beta_2$, which is the level of the control cities after the intervention. β_2 is then the trend of the control. It's how much the control grows from the pre- to the post-intervention period.

As a recap, β_1 is the increment you get by going from the control to the treated, β_2 is the increment you get by going from the pre- to the post-treatment period. Finally, if you turn both treated and post dummies on, you get $\beta_0 + \beta_1 + \beta_2 + \beta_3$. This is the level of the treated cities after the intervention, which means that β_3 is the increment in the outcome that you get by going from treated to control cities and from pre- to post-intervention period. In other words, it is the difference-in-differences estimator:

```
In [9]: import statsmodels.formula.api as smf

        smf.ols(
            'downloads ~ treated*post', data=did_data
        ).fit().params["treated:post"]
```

```
Out[9]: 0.691735953640
```

Diff-in-Diff with Fixed Effects

Yet another way to understand DID is with time- and unit-fixed effect model (two-way fixed effects or TWFE). In this model, you have treatment effect τ, unit- and time-fixed effects, α_i and γ_t, respectively:

$$Y_{it} = \tau W_{it} + \alpha_i + \gamma_t + e_{it}.$$

In order to declutter, I'm using $W_{it} = D_i Post_t$ here.

If you estimate this model, the parameter estimate associated with W will match the DID estimate you got earlier and recover the ATT. To do so, recall from Chapter 4 that you can estimate fixed effects by using dummies or by de-meaning the data. Here, for the sake of simplicity, let's just use the dummies approach. That is, let's include city and period dummies with `C(city)` and `C(post)`. Also, you need to create W by multiplying the treated and the post dummies. Just remember that the `*` operator creates the interaction between two terms and the terms by themselves. Since you only want the interaction, you need the `:` operator:

```
In [10]: m = smf.ols('downloads ~ treated:post + C(ci
                      data=did_data).fit()

         m.params["treated:post"]
```

```
Out[10]: 0.691735953640
```

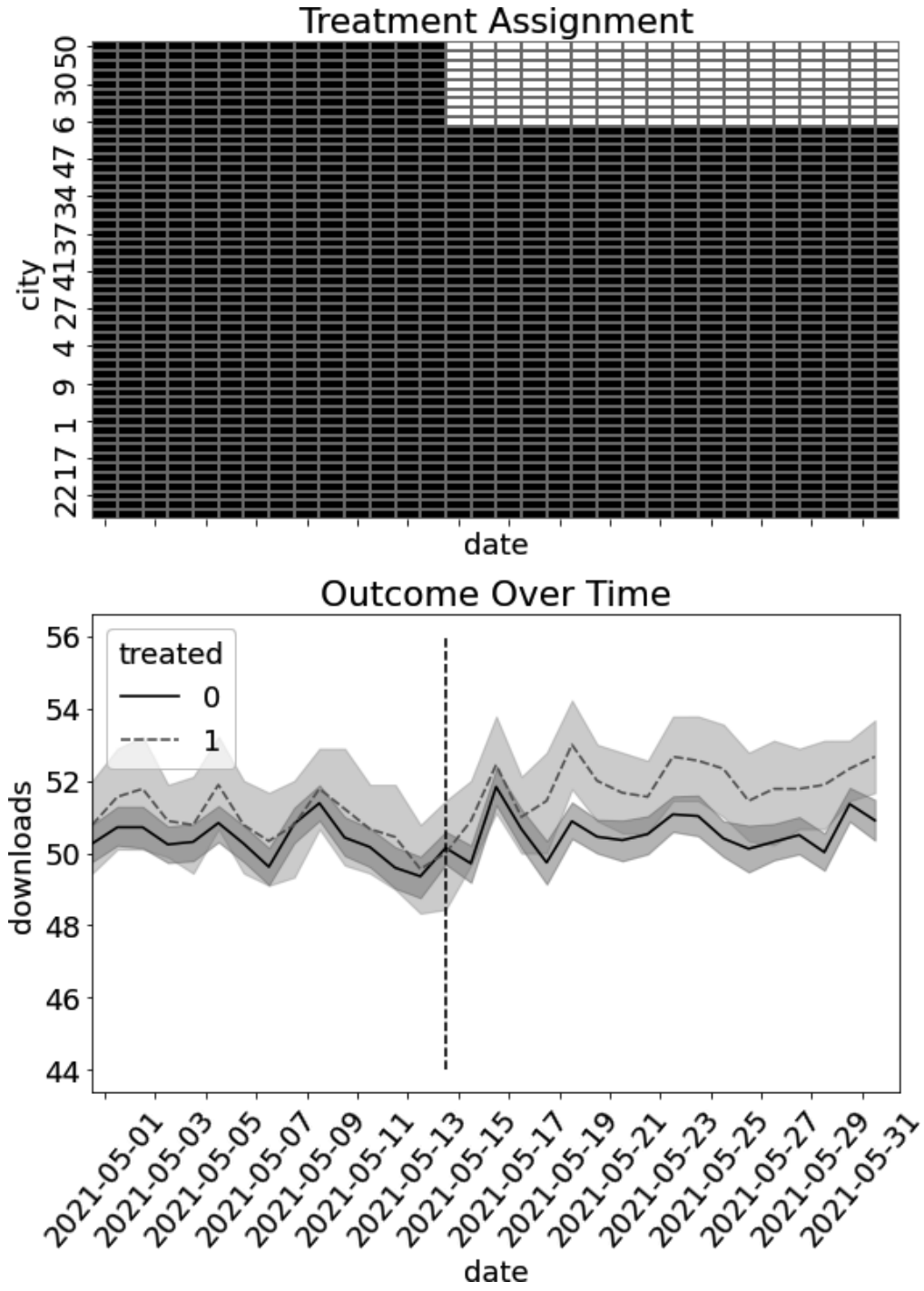
Once again, you get the exact same parameter estimate.

Multiple Time Periods

The canonical DID setting requires you to have only four data cells: pre- and post-intervention, treated and control groups. But it doesn't require that the pre and post time periods be aggregated into a single block. Canonical DID only requires that you have what is called a block design: a group of units that are never treated and a group of units that are eventually treated *at the same time period*. That is, you

can’t have the treatment rolling out to units at different moments (you’ll learn about that shortly). The marketing example you are working with has exactly this format, which means you don’t have to aggregate it by a pre- and a post-treatment period. You can just use it as is.

You can visualize this block design in the following figure, which plots the treatment assignment for each city over time. It also shows the evolution of the outcome across time so that you can get a better feeling of what DID is looking at. Namely, it is trying to see if the difference between treated and control groups increases after the intervention takes place:



To get the DID estimate with this disaggregated data, you can use the exact same formulas as before. That is, you can either regress the outcome on a treated and post dummies and the interaction between them:

```
In [11]: m = smf.ols('downloads ~ treated*post', data

         m.params["treated:post"]
```

```
Out[11]: 0.6917359536407226
```

or you can use the fixed effect specification:

```
In [12]: m = smf.ols('downloads ~ treated:post + C(ci
                     data=mkt_data).fit()

         m.params["treated:post"]
```

```
Out[12]: 0.691735953640
```

I've just shown a bunch of ways to get the exact same DID estimate. By doing so, I hope you can pool insights from all of them, increasing your chances of understanding what is going on. But if you look carefully, I've deliberately hidden the confidence intervals from the regressions you just ran. That's because the confidence intervals from those regressions are probably wrong.

Inference

I said probably wrong because, in all honesty, doing inference with panel data is incredibly tricky. There has been a lot of recent research on the topic, which is of course nice, but it also highlights that it is something we as a field are still learning how to do. The issue here is that you have $N \cdot T$ data points, but they are not independent and identically distributed, since the same unit appears multiple times. In fact, the treatment is assigned to the unit, not to the time period, so you can argue that your sample size is actually just N, not $N \cdot T$, even though this last one is what your regression will consider when computing standard errors.

To correct the overly optimistic standard errors from your regression, you can cluster the standard errors by the unit (cities, in our example):

```
In [13]: m = smf.ols(
             'downloads ~ treated:post + C(city) + C(
         ).fit(cov_type='cluster', cov_kwds={'groups'

         print("ATT:", m.params["treated:post"])
         m.conf_int().loc["treated:post"]
```

```
Out[13]: ATT: 0.6917359536407017
```

```
Out[13]: 0    0.296101
         1    1.087370
         Name: treated:post, dtype: float64
```

Clustering the errors will give you wider confidence intervals than no clustering at all:

```
In [14]: m = smf.ols('downloads ~ treated:post + C(ci
                     data=mkt_data).fit()

         print("ATT:", m.params["treated:post"])
         m.conf_int().loc["treated:post"]
```

```
Out[14]: ATT: 0.6917359536407017
```

```
Out[14]: 0    0.478014
         1    0.905457
         Name: treated:post, dtype: float64
```

Additionally, look what happens when you replace the daily data frame, `mkt_data`, from the one that you've aggregated by unit and pre- and post-treatment periods:

```
In [15]: m = smf.ols(
             'downloads ~ treated:post + C(city) + C(
         ).fit(cov_type='cluster', cov_kwds={'groups'

         print("ATT:", m.params["treated:post"])
         m.conf_int().loc["treated:post"]
```

```
Out[15]: ATT: 0.6917359536407091
```

```
Out[15]: 0    0.138188
         1    1.245284
         Name: treated:post, dtype: float64
```

The confidence interval gets even wider! This just shows that, even though the sample size should come from the units and not from the time periods, having more time periods per unit clusters can decrease the variance.

As you'll see later in this chapter, some noncanonical flavors of DID won't have a standard way to compute confidence intervals. In those situations, you can choose to bootstrap the entire estimation procedure. You just need to be a bit careful here. Since you have repeated units, the model's error for the same unit will be correlated. Hence, you need to sample (with replacement) the entire unit. This procedure is called *block bootstrap*. To implement it, you first need to write a function that samples units with replacement:

```
In [16]: def block_sample(df, unit_col):

             units = df[unit_col].unique()
             sample = np.random.choice(units, size=le

             return (df
                     .set_index(unit_col)
                     .loc[sample]
```

```
                        .reset_index(level=[unit_col]))
```

Once you have this function, you can adapt the bootstrap code from Chapter 5 to implement the block bootstrap:

```
In [17]: from joblib import Parallel, delayed

         def block_bootstrap(data, est_fn, unit_col,
                             rounds=200, seed=123, pc
             np.random.seed(seed)

             stats = Parallel(n_jobs=4)(
                 delayed(est_fn)(block_sample(data, u
                 for _ in range(rounds))

             return np.percentile(stats, pcts)
```

Finally, just to check if everything is working as expected, you can use this function to calculate the 95% CI for the DID estimator applied to the marketing data. The resulting CI is pretty similar to the one you got earlier, with standard errors clustered by units. This is a good indicator that your block bootstrap function is working:

```
In [18]: def est_fn(df):
             m = smf.ols('downloads ~ treated:post +
                         data=df).fit()
             return m.params["treated:post"]
```

```
            block_bootstrap(mkt_data, est_fn, "city")
```

```
Out[18]: array([0.23162214, 1.14002646])
```

Before ending this section, I want to warn you that, although very convenient, there are some issues with block bootstrap. For instance, if the number of treated units is small, you might end up with a sample with no treated units. Again, inference with panel data is a complex topic that I feel we don't have a clear answer to yet.

SEE ALSO

If you want to learn more about it, you can check out the paper "When Should You Adjust Standard Errors for Clustering" (2022-09) from Abadie, Athey, Imbens, and Wooldridge, four fantastic researchers in the causal inference field. I also recommend you check some alternative ways to do inference, like the one outlined in the paper "An Exact and Robust Conformal Inference Method for Counterfactual and Synthetic Controls," by Victor Chernozhukov et al.

Identification Assumptions

As you probably know by now, causal inference is a constant interaction between statistical tools and assumptions. In this chapter, I chose to begin with the statistical tool, showing how DID could leverage unit and time relationships to estimate the treatment effect. That gave you a concrete example to hold on to. Now, it's time to dig a little deeper into what kind of assumptions you were making when using DID, sometimes even without realizing it.

Parallel Trends

Previously in this book, when working with cross-sectional data, a key identification assumption was that the treatment was independent of the potential outcomes, conditioned on observed covariates. DID makes a similar, but weaker assumption: *parallel trends*.

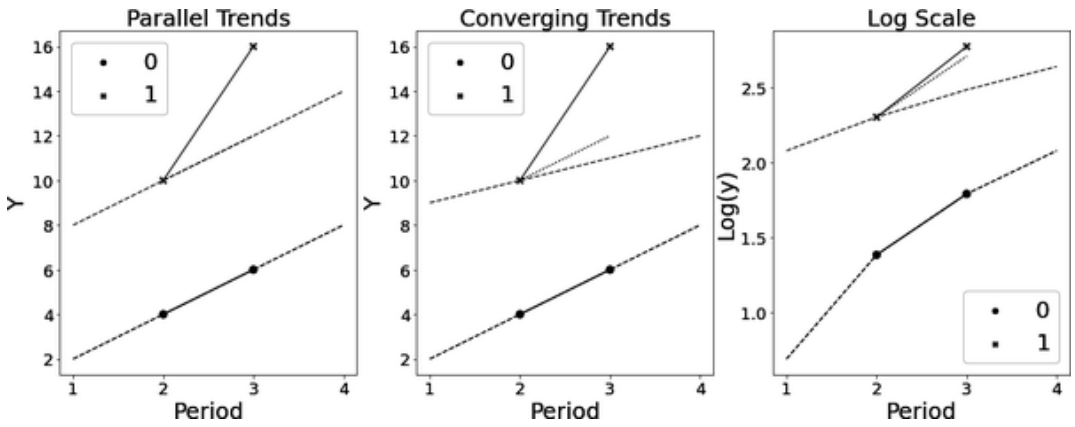
If you think about it, the DID estimator is quite intuitive when it comes to leveraging time and unit correlations. If all you had was units (no time dimension), you would have to use the control to estimate $Y(0)$ for the treated group. On the other hand, if you had a time dimension, but no control group (all units were treated at some point in time), you would have to use past $Y(0)$ from the treated units in a sort of before and after comparison. Both approaches require pretty strong assumptions. You either have to assume that the outcome of the control can identify $E[Y(0)|D = 1, Post = 1]$, which is only plausible if treated and control are comparable (like in a RCT), or if the outcome of the treated unit is a flat line across time, in which case you could use the past outcome of the treated units to identify $E[Y(0)|D = 1, Post = 1]$. In contrast, difference-in-differences makes a weaker assumption: that *the trajectory of outcomes across time would be the same, on average, for treatment and control groups, in the absence of the treatment*. It assumes that the trends in $Y(0)$ are parallel:

$$E\big[Y(0)_{it=1} - Y(0)_{it=0}\big|D = 1\big] = E\big[Y(0)_{it=1} - Y(0)_{it=0}\big|D = 0\big]$$

This assumption is untestable because it contains a term that is nonobservable: $E\big[Y(0)_{it=1}\big|D = 1\big]$. Still, for the sake of understanding it, let's pretend for a moment you could observe the $Y(0)$ potential outcome for all time periods. In the following plots, I'm representing them as dashed lines. Here you can see the potential

outcome $Y(0)$ for four periods, for both treatment and control. Additionally, each plot has four points representing the observed data for the treated and control groups and the DID estimated trajectory of the treated under the control, represented as a dotted line. The difference between this dotted line and the post-treatment outcome for the treated group is the DID estimate for the ATT.

The true ATT, however, is the difference between the post-treatment outcome for the treated group, but with respect to the dashed gray line, which represents the unobserved $Y(0)$ for the treated group:



In the first plot, the estimated and actual $Y(0)|D=1$ coincide. In this case, the parallel trend assumption is satisfied and the DID estimator recovers the true ATT. In the second plot, the trends converge. The estimated trend is steeper than the actual trend for $Y(0)|D=1$. As a result, the DID estimate would be downward biased: the difference between $E[Y(1)|D=1, Post=1]$ and the estimated trend is smaller than the difference between $E[Y(1)|D=1, Post=1]$ and the actual, but unobservable, $E[Y(0)|D=1, Post=1]$.

Finally, the last plot shows how the *parallel trends assumption is not scale invariant*. This plot simply takes the data from the first plot and

applies the log transformation to the outcome. This transformation takes a trend that was parallel and makes it convergent. I'm showing this to warn you to be very careful with DID. For instance, if you have level data, but want to measure the effect as a percent change, converting the outcome to a percentage can mess up your trends.

SEE ALSO

In the paper "When Is Parallel Trends Sensitive to Functional Form?" Jonathan Roth and Pedro Sant'Anna derive a more strict version of parallel trends that is invariant to monotonic transformation of the outcome and discuss in which situation that assumption is plausible.

An alternative way to think about the parallel trends assumption is in relation to the conditional independence assumption (CIA). While the CIA states that the level of $Y(0)$ is the same, on average, in the treated and control groups, parallel trends states that the *growth of* $Y(0)$ *is the same between treated and control groups*. This can be expressed in terms of the ΔYs you saw earlier:

$$(\Delta y_0, \Delta y_1) \perp T$$

Here lies the power of panel data: even if the treatment is not randomly assigned, so long as the treated and control groups have the same counterfactual growth, the ATT can be identified.

Just like with the independence assumption, you can relax the parallel trend assumption to be conditioned on covariates. That is, given a set of pre-treatment covariates X, the trend in $Y(0)$ is the same between treated and control group. You'll see later in this chapter how to incorporate covariates in DID.

No Anticipation Assumption and SUTVA

If the parallel trend assumption can be seen as a panel data version of the independence assumption, the no anticipation assumption is

more related to the stable unit of treatment value assumption (SUTVA). Recall that SUTVA violations happen when the effect spills over from treatment into the control units (or vice versa)? Well, here it is the same thing, but across time periods: you don't want the effect to spill over to periods when the treatment hasn't yet taken place.

If you think there is no way this can happen, consider you are trying to estimate the effect of Black Friday on sales of cell phones. If you try this, you'll see that many businesses anticipate the Black Friday discount, knowing that the period prior to Black Friday is one where customers are already shopping for products. This will likely cause you to see a spike in sales before the treatment (Black Friday) even takes place.

The fact that you have to worry about time spillover doesn't mean you don't have to worry about unit spillover. Good old SUTVA is still a big issue in panel data analysis, especially when the unit is a geographic region. That's because people are constantly moving across the geographic borders, which makes the treatment likely to spill out of the treated units.

SPATIAL SPILLOVER

Much like with everything in the panel data literature, dealing with spatial spillover is something the causal inference community is still learning about. A very good paper on this subject is “Difference-in-Differences Estimation with Spatial Spillovers,” by Kyle Butts. The paper is not hard to read and the proposed solution is easy to implement. The basic idea is to expand the two-way fixed effect formulation of DID with a dummy, S which is 1 if a control unit is deemed close enough to a treated unit:

$$Y_{it} = \tau_{it} W_{it} + \eta_0 S_i (1 - W_{it}) + \eta_1 S_i W_{it} + \alpha_i + \gamma_t + e_{it}$$

Strict Exogeneity

The strict exogeneity assumption is a pretty technical assumption that is usually stated in terms of the fixed effect model’s residuals:

$$Y_{it} = \alpha_i + X_{it}\beta + \epsilon_{it}$$

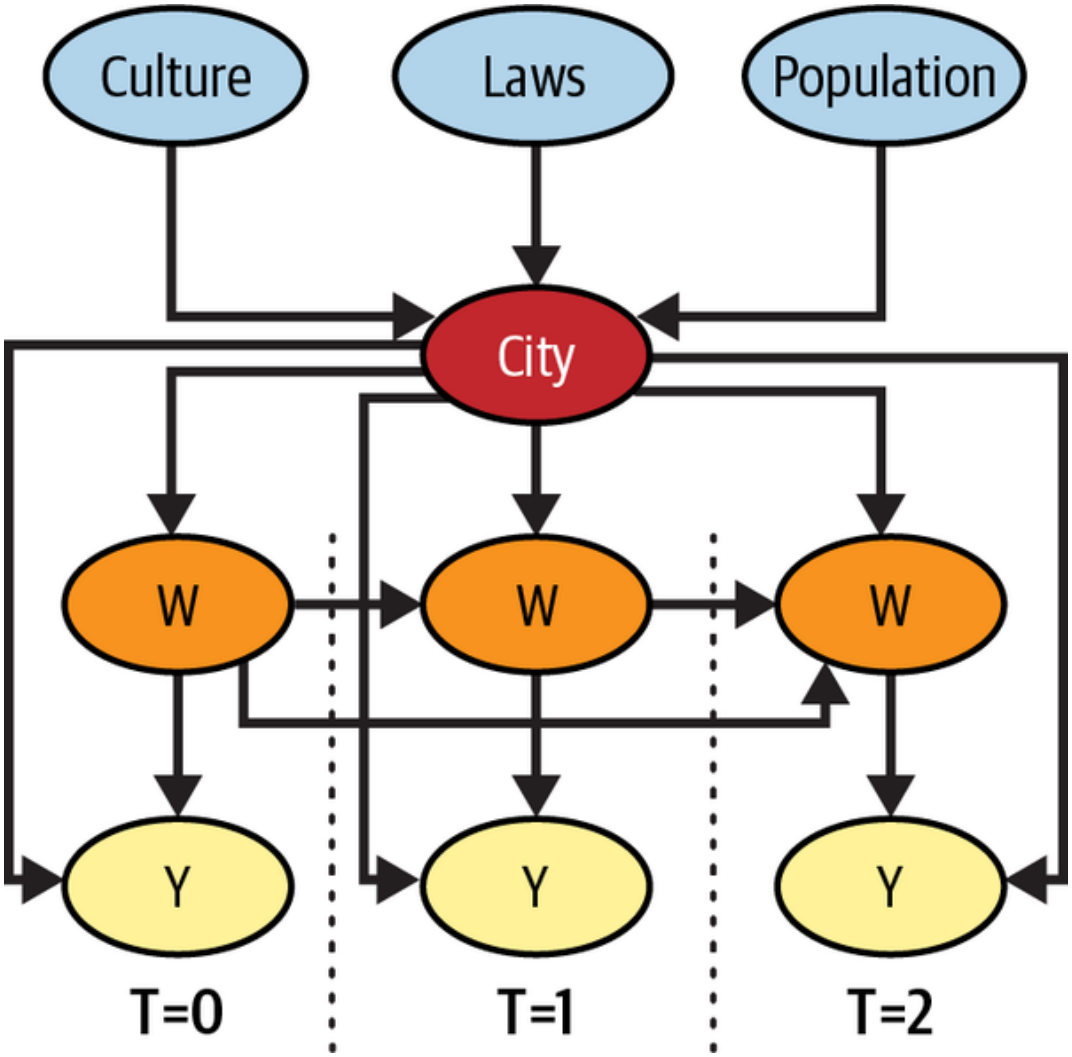
Strict exogeneity states that:

$$E[\epsilon_{it} | X_{it}, \alpha_i] = 0$$

This assumption is stronger and implies parallel trends. It is also fairly obscure, so I think it is best if we talk about it in terms of what it implies:

1. No time varying confounders
2. No feedback
3. No carryover effect

You can also make this assumption more palpable by showing it in a DAG:



Now, let's walk through what it really means.

No Time Varying Confounders

Let me start with some good news. Do you remember how I mentioned that panel data can utilize time and unit correlations? It's worth noting that having repeated observations over time can help you *identify the causal effect even when unobserved confounders are present*. This is true as long as those confounders are constant over

time or across all units. To understand this better, let's revisit the marketing example. Each city has its unique culture, laws, and population, all of which can significantly influence both the treatment and outcome variables. Some of these variables, such as culture and laws, are challenging to quantify, making them unobserved confounders that you need to account for. However, how can you do that when you can't measure them?

The trick is to see that, by zooming in on a unit and tracking how it evolves over time, you are already controlling for anything that is fixed over time. That includes any time-fixed confounders, even those that are unmeasured. In the marketing example, if downloads increase over time in a particular city, you know it cannot be due to any change in the city culture (at least not in a short time frame), simply because that confounder is fixed over time. The bottom line is that even though you can't control for time-fixed confounders, since you can't measure it, you can still block the backdoor path that goes through it, if you control for the unit itself.

If you are more of a math person, you can also see how the process of demeaning the data wipes out any time-fixed covariate. Recall that adding unit-fixed effects can be achieved by adding unit dummies, but also by computing the average of both outcome and treatment by unit and subtracting that from the original variables:

$$\ddot{Y}_{it} = Y_{it} - Y_i$$

$$\ddot{W}_{it} = W_{it} - W_i$$

Here, I'm using $W_{it} = D_i Post_t$ to denote the treatment, since D_i is time fixed. With de-meaning, any unobserved U_i vanishes. Since U_i is constant across time, you have that $U_i = U_i$, which makes $\ddot{U}_{it} = 0$

everywhere. In plain terms, unit-fixed effects wipe out any variable that is constant across time.

I'm focusing on unit-fixed effects, but a similar argument can be used to show how time-fixed effects can wipe out any variable that is fixed across units, but changes in time. In our example, those could be the country's exchange rate or inflation. Since those are nationwide variables, they are the same for all the cities.

Of course, if the unobserved confounder changes over time and unit, there is not much you can do here.

No Feedback

You might have noticed that the previous graph has another vital assumption in it. Specifically, there are no arrows extending from past outcomes, Y_{it-1}, toward current treatment, W_{it}. In other words, there is no feedback. This implies that the treatment cannot be decided based on the outcome trajectory. To illustrate, imagine the treatment was a vector indexed by time $W = (w_0, w_1, \ldots, w_T)$. In this scenario, the entire vector would have to be decided on one go. This is plausible in block designs like the one you saw before, where the treatment turns on at a particular time period and continues indefinitely. However, even then the no feedback assumption could be violated. For instance, suppose that the marketing team decided that they would do an offline marketing campaign whenever a city reached 1,000 downloads. This would violate the no feedback assumption.

SEQUENTIAL IGNORABILITY

If you want to be able to condition on past outcomes, you need to look into methods that work under sequential ignorability. Sadly, you can either control for past outcomes or time-fixed confounders, but not both. For more about this topic, I suggest you check out the paper "Causal Inference with Time-Series Cross-Sectional Data: A Reflection," by Yiqing Xu, or the book *Causal Inference* by M.A. Hernán and J.M. Robins.

No Carryover and No Lagged Dependent Variable

Beyond no feedback, you might observe that the graph also assumes *no carryover effect*, since there are no arrows from past treatment to current outcomes. Fortunately, this assumption can be relaxed if you expand the model, including lagged versions of the treatment. For example, if you believe that treatment at period $t-1$ impacts the outcome at time t, you can use the following model:

$$Y_{it} = \tau_{it} W_{it} + \theta W_{it-1} + \alpha_i + \gamma_t + e_{it}.$$

Finally, the graph also assumes no lagged dependent variable, meaning that past outcome doesn't directly cause current outcome. Lucky for you, this assumption is not really necessary; adding arrows from past Ys to future Ys doesn't hinder identification.

SEE ALSO

Representing panel data in DAG is not something trivial, but there are two great papers that try to do so. They really helped me understand what strict exogeneity implied. First, the paper "When Should We Use Unit Fixed Effects Regression Models for Causal Inference with Longitudinal Data?" by Imai and Kim. Second, the paper "Causal Inference with Time-Series Cross-Sectional Data: A Reflection," by Yiqing Xu.

Effect Dynamics over Time

By now you probably have a pretty decent understanding about canonical DID, which means you can now try to diverge from it into its more apocryphal flavors. A slightly more complicated approach is when you want to incorporate effect dynamics over time. If you look back at the plot that shows outcome evolution, you can see that the difference between treated and control group doesn't increase right after the treatment takes place. Instead, it takes some time for the treatment to reach its full effect. In other words, the treatment effect is not instantaneous. This is a fairly common phenomenon not only in marketing, but with any sort of intervention on entire geographies. It also means that you might be underestimating the final treatment effect, because you are including periods where it hasn't fully matured yet.

One simple way around this problem is to estimate the ATT over time. If you are really clever, you can achieve this by creating dummies for each post-treatment period, but my favorite way of getting effects over time involves using a bit of brute force: iterating over all the time periods and running DID as if only that period was the post-treatment one.

In order to do that, let's create a function that takes a data frame and a date and runs DID as if that date was the post treatment period:

```
In [19]: def did_date(df, date):
             df_date = (df
                        .query("date==@date |  post==0
                        .query("date <= @date")
                        .assign(post = lambda d: (d["

             m = smf.ols(
                 'downloads ~ I(treated*post) + C(cit
```

```
    ).fit(cov_type='cluster', cov_kwds={'gro

    att = m.params["I(treated * post)"]
    ci = m.conf_int().loc["I(treated * post)

    return pd.DataFrame({"att": att, "ci_low
                         index=[date])
```

First, this function filters only the pre-treatment period and the passed date. Next, it filters the dates equal to or before the passed date. If the passed date is from the post-treatment period, this filter is innocuous. If the date is from the pre-treatment period, it tosses away the dates after it. This allows you to run DID even for the period before the treatment. Actually, to do that, you need the next line of code, where the function reassigns the post-treatment period to be the specified date. Now, if you pass a date from the pre-treatment, the function will pretend that it comes from the post-treatment period, in what is sort of a placebo test in the time dimension. Finally, the function estimates a DID model, extracting the ATT and the confidence interval around it. It then stores everything in a data frame with a single line.

This function works for a single date. To get the effect for all possible dates, you can iterate over them, getting the DID estimate each time. Just keep in mind that you need to skip the first date, as you need at least two time periods for DID to run. If you store all the results in a list, you can call `pd.concat` on that list to merge all the results into a single data frame:

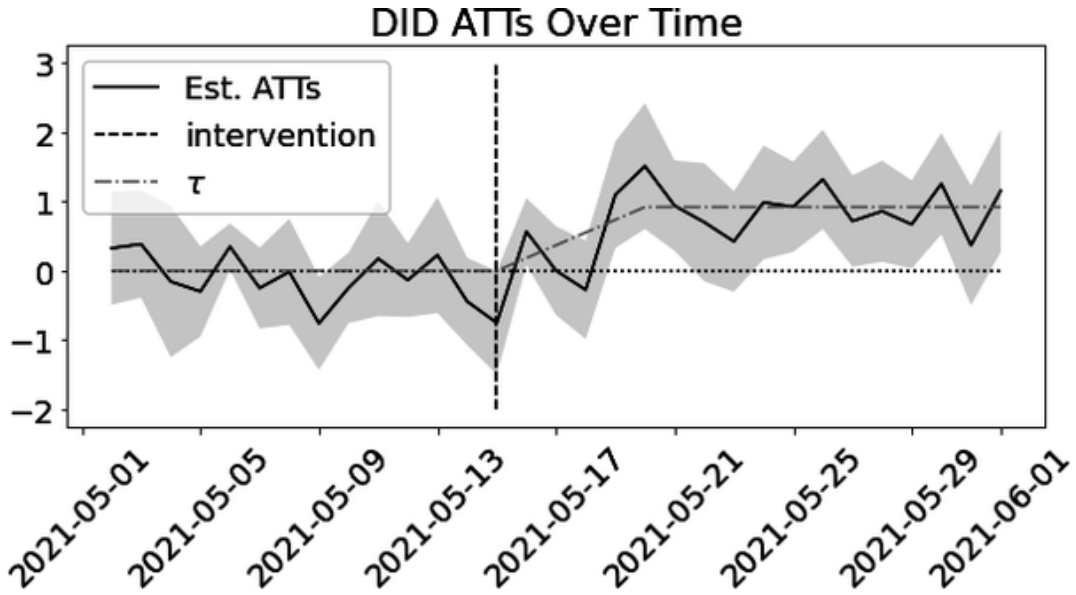
```
In [20]: post_dates = sorted(mkt_data["date"].unique(

         atts = pd.concat([did_date(mkt_data, date)
                           for date in post_dates])

         atts.head()
```

	att	ci_low	ci_up
2021-05-02	0.325397	-0.491741	1.142534
2021-05-03	0.384921	-0.388389	1.158231
2021-05-04	-0.156085	-1.247491	0.935321
2021-05-05	-0.299603	-0.949935	0.350729
2021-05-06	0.347619	0.013115	0.682123

You can then plot the effect over time alongside its confidence interval. This plot shows that the effect doesn't climb after the treatment takes place. It also seems like the ATT is a bit higher if you discard the early periods, where it hasn't fully matured yet. I'm also plotting the true effect τ so you can see how this approach manages to recover it pretty well:



The pre-treatment part of this plot also deserves your attention. During this period, all estimated effects are indistinguishable from zero, which indicates that the effect does not occur prior to treatment. This provides strong evidence that the no-anticipation assumption may be valid in this case.

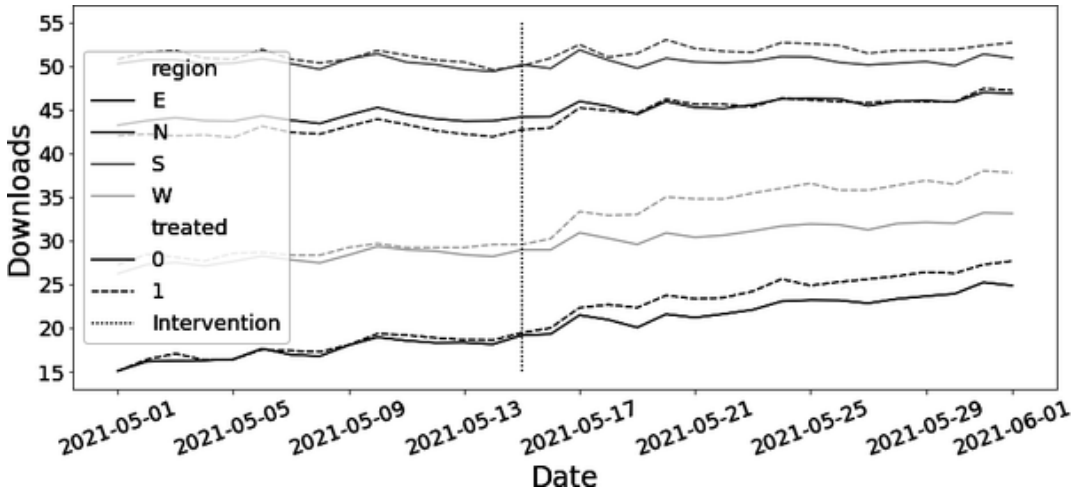
Diff-in-Diff with Covariates

Another variation of DID you need to learn is how to include pre-treatment covariates in your model. This is useful in case you suspect that parallel trend doesn't hold, but conditional parallel trend does:

$$E\big[Y(0)_{it=1} - Y(0)_{it=0} \big| D = 1, X\big] = E\big[Y(0)_{it=1} - Y(0)_{it=0} \big| D = 0, X\big]$$

Consider this situation: you have the same marketing data as before, but now, you have data on multiple regions of the country. If you plot the treatment and control outcome for each region, you'll see something interesting:

```
In [21]: mkt_data_all = (pd.read_csv("./data/short_of
                           .astype({"date":"datetime64[
```



The pre-treatment trends seem to be parallel *within a region*, but not across regions. As a result, if you simply run the two-way fixed effect specification of DID here, you'll get a biased estimate for the ATT:

```
In [22]: print("True ATT: ", mkt_data_all.query("trea

         m = smf.ols('downloads ~ treated:post + C(ci
                     data=mkt_data_all).fit()

         print("Estimated ATT:", m.params["treated:po
```

```
Out[22]: True ATT:  1.7208921056102682
         Estimated ATT: 2.068391984256296
```

Somehow, you need to account for the different trends in each region. You might think that simply adding the region as an extra covariate in the regression will solve the problem. But think again! Remember how using unit-fixed effects wipes out the effect of any time-fixed covariate? This is true not only for unobservable confounders, but also for the region covariate, which is constant across time. The end result is that naively adding it to the regression is innocuous. You'll get the same result as before:

```
In [23]: m = smf.ols('downloads ~ treated:post + C(ci
                     data=mkt_data_all).fit()
         m.params["treated:post"]
```

```
Out[23]: 2.071153674125536
```

To properly include pre-treatment covariates in your DID model, you need to recall that DID works by estimating two important pieces: the treated baseline and the control trend. It then projects the control trend into the treated baseline. This means you have to estimate the control trend for each region separately. The overkill way of doing this is to simply run a separate difference-in-differences regression for each region. You could literally loop through the regions or interact the entire DID model with region dummies:

```
In [24]: m_saturated = smf.ols('downloads ~ (post*tre
         data=mkt_data_all).fit()

atts = m_saturated.params[
```

```
        m_saturated.params.index.str.contains("post:tr
        ]atts
```

```
Out[24]: post:treated           1.676808
post:treated:C(region)[T.N]    -0.343667
post:treated:C(region)[T.S]    -0.985072
post:treated:C(region)[T.W]     1.369363
dtype: float64
```

Just keep in mind that the ATT estimates should be interpreted with respect to the baseline group, which in this case is the East region. So, the effect on the North region is 1.67–0.34, the effect on the South region is 1.67–0.98, and so on. Next, you can aggregate the different ATTs using a weighted average, where the number of cities in a region is the weight:

```
In [25]: reg_size = (mkt_data_all.groupby("region").s
                     /len(mkt_data_all["date"].unique

         base = atts[0]

         np.array([reg_size[0]*base]+
                  [(att+base)*size
                   for att, size in zip(atts[1:], reg
                 ).sum()/sum(reg_size)
```

```
Out[25]: 1.6940400451471818
```

Even though I said this is overkill, it is actually a pretty good idea. It is very easy to implement and hard to get it wrong. Still, it has some problems. For instance, if you have many covariates or a continuous covariate, this approach will be impractical. Which is why I think you should know that there is another way. Instead of interacting the region with both post and treated dummies, you can interact with the post dummy alone. This model will estimate the trend (pre- and post-outcome levels) for the treated in each region separately, but it will fit a single intercept shift to the treated and post period:

```
In [26]: m = smf.ols('downloads ~ post*(treated + C(r
                     data=mkt_data_all).fit()

         m.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.(
Intercept	17.3522	0.101	172.218	0.000	17
C(region)[T.N]	26.2770	0.137	191.739	0.000	26
C(region)[T.S]	33.0815	0.135	245.772	0.000	32
C(region)[T.W]	10.7118	0.135	79.581	0.000	10
post	4.9807	0.134	37.074	0.000	4.7
post:C(region)[T.N]	-3.3458	0.183	-18.310	0.000	-3.
post:C(region)[T.S]	-4.9334	0.179	-27.489	0.000	-5.
post:C(region)[T.W]	-1.5408	0.179	-8.585	0.000	-1.
treated	0.0503	0.117	0.429	0.668	-0.
post:treated	1.6811	0.156	10.758	0.000	1.3

The parameter associated with `post:treated` can be interpreted as the ATT. It is not exactly the same ATT that you got before, but it is pretty close. The difference appears because—as you should know by now—regression averages the regions ATT by variance, while before, you averaged them by region size. This means that regression will overweight regions where the treatment is more evenly distributed (has higher variance).

This second approach is much faster to run, but the downside is that it requires careful thinking on how you go about doing the interactions. For this reason, I recommend you use it only if you really know what you are doing. Or, before you use it, try to build some simulated data where you know the true ATT and see if you can

recover it with your model. And remember: there is no shame in just running a DID model for each region and averaging the results. In fact, it is a particularly clever idea.

Doubly Robust Diff-in-Diff

Another way of incorporating pre-treatment and time invariant covariates to allow for conditionally parallel trends is by making a doubly robust version of difference-in-differences (DRDID). To do this, you can take a lot of the ideas from Chapter 5, when you learned how to craft a doubly robust estimator. However, you'll need to make some adjustments. First, instead of having a raw outcome model, since DID works with Δy, you'll also need a model for the delta outcome over time. Second, since you only care about the ATT, you just need to reconstruct the treated population from the control units. All of this will make more sense as I show you the steps to build DRDID.

Propensity Score Model

The first step in DRDID is a propensity score model $\hat{e}(X)$ that uses the pre-treatment covariates to *estimate the probability that a unit comes from the treated group*. This model doesn't care about the time dimension, which means you only need one period worth of data to estimate it:

```
In [27]: unit_df = (mkt_data_all
                    # keep only the first date
                    .astype({"date": str})
                    .query(f"date==' {mkt_data_all[' da
                    .drop(columns=["date"])) # just t
```

```
        ps_model = smf.logit("treated~C(region)", da
```

Delta Outcome Model

Next, you need the outcome model for Δy, which means that first you need to construct the delta outcome data. To do that you need to take the difference between the average outcome at the pre- and post-treatment periods. Once you do that, you'll again end up with one row for each unit, since the time dimension has been differentiated out:

```
In [28]: delta_y = (
             mkt_data_all.query("post==1").groupby("c
             - mkt_data_all.query("post==0").groupby(
         )
```

Now that you have Δy, you can join it back into the unit dataset and fit the outcome model in it:

```
In [29]: df_delta_y = (unit_df
                       .set_index("city")
                       .join(delta_y.rename("delta_y"

         outcome_model = smf.ols("delta_y ~ C(region)
```

All Together Now

It's time to join all the pieces. Let's start by gathering all the data you need into a single data frame. For the final estimator, you'll need the actual Δy, the propensity score, and the delta outcome prediction. For that, you can start from the `df_delta_y` you used to build your outcome model and make predictions using both the propensity score model, $\hat{e}(x)$, and the outcome model, $\hat{m}(x)$. The result is, once more, a unit-level data frame:

```
In [30]: df_dr = (df_delta_y
                  .assign(y_hat = lambda d: outcome_m
                  .assign(ps = lambda d: ps_model.pre

         df_dr.head()
```

	region	treated	tau	d
city				
1	W	0	0.0	2
2	N	0	0.0	4
3	W	0	0.0	3
4	W	0	0.0	2
5	S	0	0.0	5

With that, let's think about what a doubly robust DID would look like. As with all DID, the ATT estimate is the difference between the trend, had the units been treated, from the trend they would have under the control. Since those are counterfactual quantities, I'll represent them with Δy_1 and Δy_0, respectively. So, to summarize, the ATT would be given by:

$$\hat{\tau}_{DRDID} = \widehat{\Delta y}_1^{DR} - \widehat{\Delta y}_0^{DR}$$

It's not much, I'll admit, but it's a nice start. From there, you need to think how to estimate the Δy_Ds in a doubly robust manner.

Let's focus on Δy_1. To estimate the treated counterfactual you would weight $y - \widehat{m}(x)$ by the inverse of the propensity score, which would reconstruct y_1 for the entire population (see Chapter 5). Here, since

you only care about the ATT, you don't need that; you already got the treatment population. Hence, the first term becomes:

$$\widehat{\Delta y_1}^{DR} = 1/N_{tr} \sum_{i \in tr} (\Delta y - \hat{m}(X))$$

For the other term, you would use weights $1/(1 - \hat{e}(x))$ to reconstruct the general population under the control. But again, since you care about the ATT, you need to reconstruct the treatment population under the control. To do that, you can simply multiply the weights by the chance of being a treated unit, which, conveniently, is just the propensity score:

$$w_{co} = \hat{e}(X) \frac{1}{1 - \hat{e}(X)}$$

Having defined the weight, you can use it to obtain the estimate for Δy_0:

$$\widehat{\Delta y_0}^{DR} = \sum_{i \in co} w_{co} (\Delta y - \hat{m}(X)) / \sum w_{co}$$

That is pretty much it. As (almost) always, it looks a lot simpler in code than in math:

```
In [31]: tr = df_dr.query("treated==1")
         co = df_dr.query("treated==0")

         dy1_treat = (tr["delta_y"] - tr["y_hat"]).me

         w_cont = co["ps"]/(1-co["ps"])
         dy0_treat = np.average(co["delta_y"] - co["y

         print("ATT:", dy1_treat - dy0_treat)
```

```
Out[31]: ATT: 1.6773180394442853
```

It is remarkably close to the true ATT and to the ATT you got earlier when you added covariates to DID. The advantage here is that you get two shots at getting the estimation right. The DRDID will work if either (but not necessarily both) the propensity score model or the outcome model are correct. I won't do it here to avoid making this chapter too long, but I encourage you to try replacing either the `ps` columns or the `y_hat` column by a randomly generated column and recompute the preceding estimate. You'll see that the end result will still be close to the actual one.

SEE ALSO

This method was proposed in the paper "Doubly Robust Difference-in-Differences Estimators," by Sant'Anna and Zhao. The paper has a lot more content, including how to do inference on this estimator and how to achieve double robustness when you have repeated cross-sectional data (when units can change at each time period) as opposed to a panel data (when all units are the same).

Just like when you did doubly robust estimation with cross-sectional data, to get confidence intervals for DRDID, you would need to use the block bootstrap function you implemented earlier, wrapping the entire procedure—outcome model, propensity score model, and putting it all together—in a single estimation function.

Staggered Adoption

Up until this point, the type of data you've been looking at followed a block design, with only two periods: a pre- and post-treatment. Even

though each period had multiple dates, at the end of the day, all that mattered was that you had a group of units that were all treated at the same point in time and a group of units that were never treated. This block design is the poster child to difference-in-differences analysis since it keeps things extremely simple, allowing you to estimate the baselines and trends nonparametrically—that is, by simply computing a bunch of sample averages and comparing them. But it can also be fairly limited. What if the treatment gets rolled out to the units at different points in time?

A much more common situation with panel data is the *staggered adoption design*, where you have multiple groups of units, which I'll call G, and each group gets treated at a different point in time (or never). Since the timing of the treatment is what defines a group, it's common to refer to them as a cohort: the group G that gets the treatment at time t is the cohort G_t.

Bringing this to the marketing data you've been looking at, you had two cohorts: a never treated cohort, or G_∞, and a group that got the treatment at 2021-05-15, or $G_{15/05}$. But that's only because I've hidden what happens after 2021-06-01. Now that you're ready for a more complex situation, look at the `mkt_data_cohorts` data frame, which also contains data on cities from all the regions, but now up until 2021-07-31:

```
In [32]: mkt_data_cohorts = (pd.read_csv("./data/offl
                               .astype({
                                   "date":"datetime64[n
                                   "cohort":"datetime64

         mkt_data_cohorts.head()
```

	date	city	region	c
0	2021-05-01	1	W	2
1	2021-05-02	1	W	2
2	2021-05-03	1	W	2
3	2021-05-04	1	W	2
4	2021-05-05	1	W	2

It's hard to see all the data by just looking at the top rows, but Figure 8-2 shows the treatment status across time. There you can see what a staggered adoption design looks like.

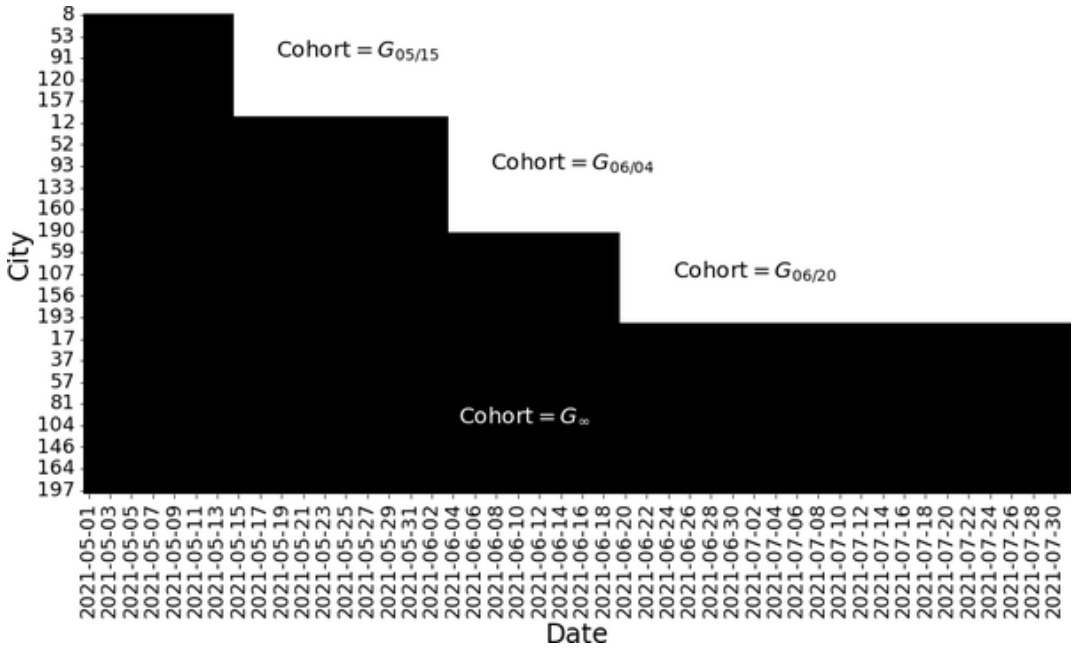


Figure 8-2. In a staggered adoption design, the treatment gets gradually rolled out to more and more units

Previously, you had data up until 2021-06-01, so it looked like you had a small treatment group and a huge never treated group. But once you expand your data, you can see that the offline marketing campaign got rolled out to other cities later on. Now, you have four different cohorts, three of which are treated and a never treated group (which has cohort 2100-01-01 in this dataset).

WARNING

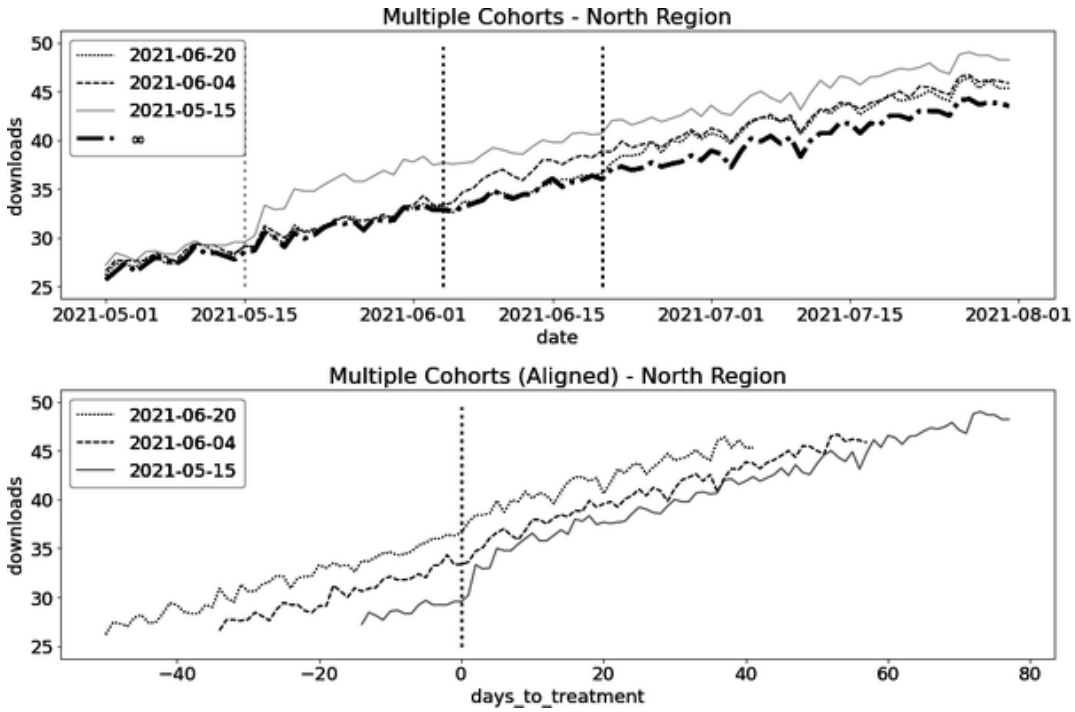
Just like with the block design, with staggered adoption you'll assume that once the treatment turns on, it stays that way forever. This is important to keep things tractable. In panel data analysis, the potential outcomes are defined by vectors representing the trajectory of outcome at each time period, $D = (Y_{d1}, Y_{d2}, \ldots, Y_{dT})$, and the treatment effect is defined by contrasting two of those trajectories. This means there are about 2^T ways to define the treatment effect if you allow the treatment to turn on and off.

To take things one bite at a time, let's forget about covariates for now, focusing only on the West region. I'll show how to handle covariates later. For now, just focus on the staggered adoption component of the problem:

```
In [33]: mkt_data_cohorts_w = mkt_data_cohorts.query(
         mkt_data_cohorts_w.head()
```

	date	city	region	c
0	2021-05-01	1	W	2
1	2021-05-02	1	W	2
2	2021-05-03	1	W	2
3	2021-05-04	1	W	2
4	2021-05-05	1	W	2

If you plot the average downloads over time for each cohort, you can see a clear picture. The outcome of $G = 05/15$ shows an increase right after the date 2021-15-05. The same is true for the cohorts $G = 06/04$ and $G = 06/20$. Meanwhile, the group that was never treated follows what appears to be a beautifully parallel trend to the treated groups prior to the treatment. Another thing to pay attention to is that the effect takes some time to mature, something you've seen before. This becomes even clearer if you plot the outcome after aligning the cohorts, which you can see in the second image:



You might argue that the preceding data is extraordinarily well behaved, which makes it obvious it was simulated. You might even be tempted to conclude that diff-in-diff will have no problem recovering the true ATT. Well, let's try it out:

```
In [34]: twfe_model = smf.ols(
             "downloads ~ treated:post + C(date) + C(
             data=mkt_data_cohorts_w
         ).fit()

         true_tau = mkt_data_cohorts_w.query("post==1

         print("True Effect: ", true_tau)
         print("Estimated ATT:", twfe_model.params["t
```

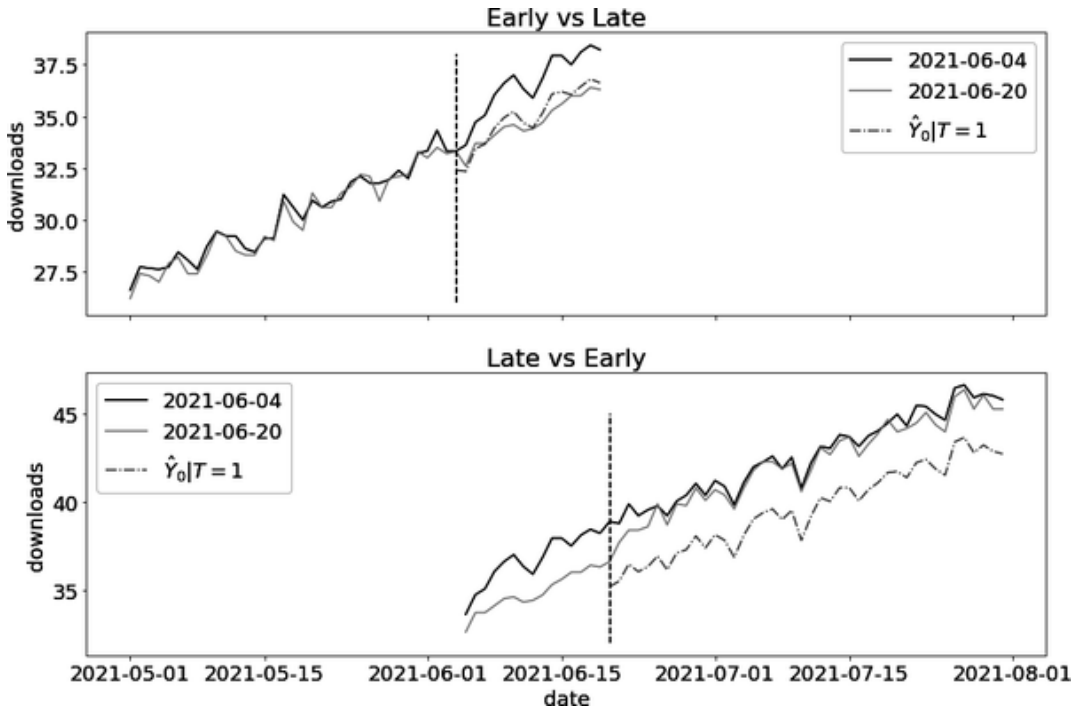
```
Out[34]: True Effect:  2.2625252108176266
         Estimated ATT: 1.7599504780633743
```

As you can see, something is off. The effects seems downward biased! What is going on here?

This issue has been at the center of the very recent literature on panel data. Unfortunately, this chapter would be too long if I tried to give you a full explanation. What I can do is give you a glimpse and, if you are interested, point you to further resources. The root of this problem lies in the fact that, when you have staggered adoption, beyond the traditional DID assumptions you saw earlier, *you also need to assume that effects are homogeneous across time*. As discussed earlier, this is not the case with this data. The effect takes some time to mature, meaning it is lower just after the treatment takes place and gradually climbs up afterward. This effect change over time causes bias in your ATT estimate.

Let's examine two groups of cities to understand why. The first group, which I'll call the *early treated* cohort, received treatment on 06/04. The second group, which I'll refer to as the *late treated* cohort, received treatment on 06/20. The two-way fixed effect model you just estimated actually uses a series of 2×2 diff-in-diff runs and combines them into a final estimate. In one of these runs, the model estimates the effect of treatment on the early treated cohort using the late treated group as a control. This is valid since the late treated cohort can be considered a not-yet-treated group. However, the model is also estimating the effect on the late treated cohort by using the early treated cohort as a control. This approach is acceptable, but only if the treatment effect is not variable across time. You can see why in

the following image. The picture shows both comparisons as well as the estimated counterfactual, Y_0. It's worth noting that the role played by each cohort reverses from one plot to the next:



As you can see, the early versus late comparison seems fine. The issue is on the late versus early. The control group (cohort 06/04) is already treated, even though it serves as a control here. Moreover, since the effect is heterogeneous, gradually climbing up, the trend in the control (early treated) is steeper than it would be, had the cohort not yet been treated. This extra steepness from the gradually increasing effect causes an overestimate in the control trend, which in turns translates to a downward-biased ATT estimate. This is why using the already treated as a control will bias your results if the treatment effect is heterogeneous across time.

SEE ALSO

Like I said, there is a bunch of recent literature on this limitation of the two-way fixed effect model. If you want to learn more, I strongly recommend the paper "Difference-in-Differences with Variation in Treatment Timing," by Andrew Goodman-Bacon. His diagnosis of the problem is very neat and intuitive. Not to mention it comes with nice pictures to help with the general understanding of the paper.

Now that you know the problem, it's time to look at the solution. Since the issue lies on effect heterogeneity, the remedy will be to use a more flexible model, one that fully takes into account those heterogeneities.

RACTICAL EXAMPLE

HIGHER EDUCATION AND GROWTH IN DEVELOPING COUNTRIES

In a more recent paper, "Higher Education Expansion, Labor Market, and Firm Productivity in Vietnam," Khoa Vu and Tu-Anh Vu-Thanh looked at the rapid increase at the number of universities in Vietnam to figure out the impact of higher education on wage. They took advantage that the higher education expansion was different for each province, which allowed them to identify the effect of universities using difference-in-differences.

> *We collect a dataset on the timing and location of university openings and estimate that individuals' exposure to a university opening increases their chances of completing college by over 30%. It also raises their wage by 3.9% and household expenditure by 14%.*

Heterogeneous Effect over Time

There is good news and bad news. First, the good news: you've identified the problem. Namely, you know that TWFE is biased when applied to staggered adoption data that has time-heterogeneous effects. In a more technical notation, your data generating process has different effect parameters:

$$Y_{it} = \tau_{it} W_{it} + \alpha_i + \gamma_t + e_{it},$$

but you were assuming that the effect was constant:

$$Y_{it} = \tau W_{it} + \alpha_i + \gamma_t + e_{it}.$$

If that is the problem, an easy fix would be to simply allow for a different effect for each time and unit. You could, in theory, achieve something like that with the following formula:

```
downloads ~ treated:post:C(date):C(city) + C(date) +
```

That's it right? Problem solved? Well, not quite. Now for the bad news: this model would have more parameters than there are data points. Since you are interacting date and unit, you would have one treatment effect parameter for each unit for each time period. But this is exactly the number of samples you have! OLS wouldn't even run here.

OK, so you need to reduce the number of treatment effect parameters of the model. To achieve that, can you think of a way of somehow grouping units? If you scratch your head a little, you can see a very natural way to group units: by cohort! You know that the effect in an entire cohort follows the same pattern over time. So, a natural improvement on that impractical model is to allow the effect to change by cohort, instead of by units:

$$Y_{it} = \tau_{gt} W_{it} + \alpha_i + \gamma_t + e_{it}.$$

That model has a more reasonable number of treatment effect parameters, since the number of cohorts is usually much smaller than the number of units. Now, you can finally run the model:

```
In [35]: formula = "downloads ~ treated:post:C(cohort

         twfe_model = smf.ols(formula, data=mkt_data_
```

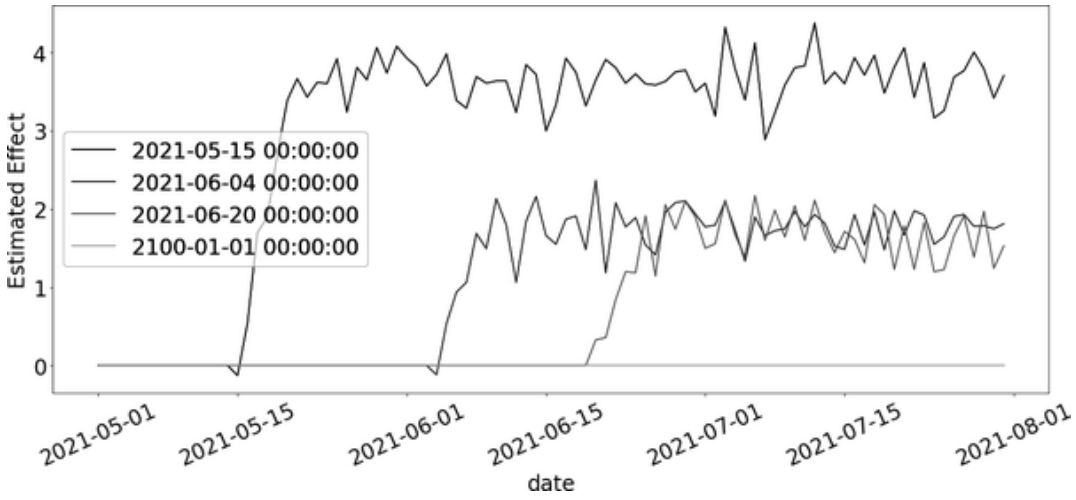
This will give you multiple ATT estimates: one for each cohort and date. So, to see if you got it right, you can compute the estimated individual treatment effect implied by your model and average out the result. To do that, just compare the actual outcome at the post-treatment period from the treated units with what your models predict for $\hat{y}_0$:

```
In [36]: df_pred = (
             mkt_data_cohorts_w
             .query("post==1 & treated==1")
             .assign(y_hat_0=lambda d: twfe_model.pre
             .assign(effect_hat=lambda d: d["download
         )

         print("Number of param.:", len(twfe_model.pa
         print("True Effect: ", df_pred["tau"].mean()
         print("Pred. Effect: ", df_pred["effect_hat"
```

```
Out[36]: Number of param.: 510
         True Effect:  2.2625252108176266
         Pred. Effect:  2.259766144685074
```

Finally! This gives you a model with a bunch of parameters (510!), but it does manage to recover the true ATT. You can even extract those ATTs and plot them:



The nice thing about this plot is that it's in accordance with your intuition of how the effect should behave: it gradually climbs up and it stays constant after a while. Also, it shows you that the effect is zero for all the pre-treatment periods and, consequently, for the never treated cohort. This might give you some idea on how to reduce the number of parameters from this model. For instance, you could only consider effects from time periods that are greater than the cohort:

$$Y_{it} = \tau_{g,t \geq g} W_{it} + \alpha_i + \gamma_t + e_{it}.$$

This would involve a nontrivial amount of feature engineering, though, as you would have to group the dates prior to the treatment, but it's good to know it is possible.

SEE ALSO

This solution to the treatment effect heterogeneity problem was inspired by the papers "Estimating Dynamic Treatment Effects in Event Studies with Heterogeneous Treatment Effects," by Sun and Abraham, and the paper "Two-Way Fixed Effects, the Two-Way Mundlak Regression, and Difference-in-Differences Estimators," by Jeffrey Wooldridge.

Just like when you approached the problem of including covariates in the diff-in-diff model, there are two types of solutions for this TWFE bias. The one you just saw involves cleverly interacting dummies when running the two-way fixed effect model. Another approach involves breaking the problem into multiple 2 × 2 diff-in-diffs, solving each one individually and combining the results. One way to do this is by estimating one diff-in-diff model for each cohort, using the never treated group as a control:

```
In [37]: cohorts = sorted(mkt_data_cohorts_w["cohort"

         treated_G = cohorts[:-1]
         nvr_treated = cohorts[-1]

         def did_g_vs_nvr_treated(df: pd.DataFrame,
                                  cohort: str,
                                  nvr_treated: str,
                                  cohort_col: str = "
                                  date_col: str = "da
                                  y_col: str = "downl
             did_g = (
                 df
                 .loc[lambda d:(d[cohort_col] == coho
                              (d[cohort_col] == nvr_
                 .assign(treated = lambda d: (d[cohor
                 .assign(post = lambda d:(pd.to_datet
             )
```

```
            att_g = smf.ols(f"{y_col} ~ treated*post
                            data=did_g).fit().params
            size = len(did_g.query("treated==1 & pos
            return {"att_g": att_g, "size": size}


        atts = pd.DataFrame(
            [did_g_vs_nvr_treated(mkt_data_cohorts_w
             for cohort in treated_G]
        )

        atts
```

	att_g	size
0	3.455535	702
1	1.659068	1044
2	1.573687	420

Then, you can combine the result with a weighted average, where the weights are the sample size ($T * N$) of each cohort. The resulting estimate is remarkably similar to what you estimated before:

```
In [38]: (atts["att_g"]*atts["size"]).sum()/atts["siz
```

```
Out[38]: 2.2247467740558697
```

Alternatively, instead of using the never treated as the control, you could use the not yet treated, which increases the sample size of the control. This is a bit more cumbersome, as you would have to run diff-in-diff multiple times for the same cohort.

SEE ALSO

This second solution to the effect heterogeneity problem was inspired by the paper "Difference-in-Differences with Multiple Time Periods," by Pedro H. C. Sant'Anna and Brantly Callaway. In the paper, they also cover how to use the not-yet treated as a control group and how to use doubly robust difference-in-differences.

Covariates

Having gotten the TWFE bias issue out the way, all there is left to do is see how to use the entire dataset, all time periods, with staggered adoption design, and all regions, which will require you to include covariates in your model.

Fortunately, there is nothing particularly new here. All you have to do is remember how you've added covariates to the diff-in-diff model earlier. In that case, you've interacted the covariates with the post treatment dummy. Here, analogous to the post-treatment dummy is the date column, which marks the passage of time. Hence, all you have to do is interact the covariates with that column:

```
In [39]: formula = """
         downloads ~ treated:post:C(cohort):C(date)
         + C(date):C(region) + C(city) + C(date)"""
```

```
            twfe_model = smf.ols(formula, data=mkt_data_
```

Once more, since this model will give you a bunch of parameter estimates, you can get the ATT by computing the individual effects and averaging them out:

```
In [40]: df_pred = (
             mkt_data_cohorts
             .query("post==1 & treated==1")
             .assign(y_hat_0=lambda d: twfe_model.pre
             .assign(effect_hat=lambda d: d["download
         )

         print("Number of param.:", len(twfe_model.pa
         print("True Effect: ",  df_pred["tau"].mean(
         print("Pred. Effect: ", df_pred["effect_hat"
```

```
Out[40]: Number of param.: 935
         True Effect:  2.078397729895905
         Pred. Effect:  2.0426262863584568
```

If you choose to break down staggered adoption into multiple 2 × 2 blocks, you could also add covariates in each DID model individually, pretty much like you did earlier.

Key Ideas

Panel data methods is an exciting and rapidly evolving field in causal inference. A lot of the promises come from the fact that having an extra time dimension allows you to estimate counterfactuals for the treated not only from the control units, but also from the treated units' past.

In this chapter, you've explored multiple ways of applying difference-in-differences. DID relaxes the traditional unconfoundedness assumption, $Y_d \perp T|X$ to the conditional parallel assumption:

$$\Delta Y_d \perp T|X$$

This gives you some hope if you have unobservable confounders. With panel data, you can still identify the ATT, as long as those confounders are constant across time (for the same unit) or across units (for the same time period).

Despite its great powers, DID doesn't come without its complexities. If you move beyond the canonical DID formulation, you need to be very careful with your modeling. While 2 × 2 DID has the flexibility of a nonparametric model, the same cannot be said about the more general staggered adoption setting, which requires you to make additional functional form assumptions.

This chapter teaches you how to deal with many extensions beyond the simple 2 × 2 case, adding covariates, estimating the effect evolution over time, and allowing different treatment timing. However, keep in mind that all of this is very new, and I wouldn't be surprised if the field moves beyond what's in here in the near future. Still, this chapter should give you a pretty solid foundation to catch up quickly, should the necessity arise.